

Laboratorio 8. DuckDB

Javier Eduardo España (23361), Angel Esteban Esquit (23221), Roberto Jose Barreda (23354)

Ejercicio 1. Preparación del ambiente

Estructura del proyecto

Ruta	Propósito
data/raw	Parquet de la TLC tal como se descargan, en una carpeta por tipo de taxi y año, y el catálogo de zonas. Git los ignora
data/processed	Bases de DuckDB del benchmark y del tablero. Git las ignora
notebooks	Consultas, resultados e interpretación de los ejercicios 3 a 8
scripts	Descarga, verificación, benchmark y tablero
sql	Consultas en una carpeta por ejercicio. vistas.sql define las vistas viajes, viajes_validos y zonas, que registran las transformaciones sobre los datos
docs	Respuestas de los ejercicios 1, 2 y 9 en Word y PDF. docs/evidencias guarda las salidas de cada ejecución. Cada archivo lleva el número de su ejercicio
Dockerfile	Imagen del servicio lab con Python 3.11 y las librerías de requirements.txt
metabase.Dockerfile	Imagen de Metabase con el driver de DuckDB, sobre Debian porque el driver necesita glibc
docker-compose.yml	Servicios lab y metabase
requirements.txt	Versiones fijas de las librerías de Python, con duckdb en la misma versión que el driver de Metabase

1.1 y 1.2 Fork y ambiente con Docker Compose

El repositorio es un fork de github.com/menene/duckdb. El ambiente se levanta desde la raíz del fork con los comandos de 1.5.

La primera construcción descarga la imagen de Python, la de Java y el jar de Metabase, cerca de 3 GB en total.

docker-compose.yml monta data, notebooks, scripts, sql y docs en /workspace del servicio lab, así que lo que se descarga o se genera dentro del contenedor queda en la carpeta local. Metabase monta data en /workspace/data para leer las bases de DuckDB.

El equipo agregó .dockerignore para que la construcción no envíe los datos a Docker. La imagen de lab copia requirements.txt y la de Metabase no copia archivos del proyecto.

1.3 Servicios

Servicio	Puerto	Uso
lab	8888	JupyterLab, Python y DuckDB
metabase	3000	Metabase con el driver de DuckDB

Laboratorio 8. DuckDB

Javier Eduardo España (23361), Angel Esteban Esquit (23221), Roberto Jose Barreda (23354)

Los puertos se publican en 127.0.0.1, por eso JupyterLab no pide token.

Prueba	Resultado
docker compose ps	lab8-lab y lab8-metabase en estado running
JupyterLab en http://localhost:8888	Responde con código 200
Metabase en http://localhost:3000/api/health	Responde ok
scripts/verificar_ambiente.py dentro de lab	Python 3.11.14 en Linux, versiones de requirements.txt, DuckDB escribe y lee Parquet, estructura completa y Metabase alcanzable desde lab

La salida de estas pruebas está en docs/evidencias/01_verificacion_ambiente.txt.

1.4 Herramientas disponibles

Herramienta	Versión	Uso
Python	3.11.14	Lenguaje de scripts y notebooks
DuckDB	1.5.5	Consultas SQL sobre Parquet y base local
pandas	3.0.6	Tablas de resultados
PyArrow	25.0.1	Soporte de Parquet y Arrow para pandas
matplotlib	3.11.2	Gráficas en los notebooks
requests	2.34.2	Descarga de archivos y llamadas a la API de Metabase
JupyterLab	4.6.4	Notebooks
Metabase	v0.63.19	Tablero de indicadores
Driver de DuckDB para Metabase	1.5.5.0	Conexión de Metabase con las bases de DuckDB
Java (Eclipse Temurin)	21	Ejecución de Metabase

1.5 Procedimiento para levantar el ambiente

Requiere Docker con Compose y Git. Los mismos pasos están en la sección Ejecución del README.

```
git clone https://github.com/bar23354/DS-Lab08.git
cd DS-Lab08
docker compose up -d --build
docker compose exec lab python scripts/verificar_ambiente.py
```

JupyterLab queda en http://localhost:8888 y Metabase en http://localhost:3000. Los servicios se detienen con docker compose down.

1.6 Por qué un ambiente reproducible

El ambiente queda definido en archivos del repositorio. Cada integrante y el docente ejecutan el proyecto con la misma versión de Python, de DuckDB, de Metabase y de cada librería, sin depender de lo instalado en su computadora. Si un resultado cambia, la causa está en los datos o en el código y no en la instalación.

En este laboratorio importa por tres razones. Los datos crecen con cada año agregado y el análisis se vuelve a ejecutar varias veces. Los tiempos del benchmark dependen de la versión de DuckDB. Y Metabase lee las bases con su propio driver, que tiene que coincidir con la versión de DuckDB que las crea.